MODULE 12 · BLOCK 3 - DATA COLLECTIONS

Lists II: references, copies and comprehensions

Two names on one list, three ways to copy, sort versus sorted, comprehensions, and lists inside lists.

BLOCK 3 of 4 EXAM WEIGHT 25% MODULE 12 of 28

admin@sthithapragna.com

Assignment **never copies** a list. It sticks another name on the same list. To get a separate list, you must copy it.

b = a sticks a second label on the same list

```
1  a = [1, 2, 3]
2  b = a
3  b[0] = 9
4  print(a, b is a)     # [9, 2, 3] True
```

2 No new list is made. b and a are two names for one list.

3 Changing the list through b changes what a sees, because it is the same list.

Three ways to copy a list

```
1  a = [1, 2, 3]
2  c1 = a[:]            # slice of everything
3  c2 = list(a)         # build a new list from a
4  c3 = a.copy()        # the copy method
5  c1[0] = 0
6  print(a, c1)         # [1, 2, 3] [0, 2, 3]
7  print(c2 == a, c2 is a)   # True False
```

2-4 All three build a new list holding the same elements.

5-6 Changing the copy leaves the original untouched.

7 Equal values, but a different object.

Change the list, or move the label?

CHANGE THE LIST

Seen through every name

```
1  a = [1, 2]
2  b = a
3  b.append(3)
4  print(a)
```

[1, 2, 3]. append changes the one list both names are stuck on.

MOVE THE LABEL

Only that name changes

```
1  a = [1, 2]
2  b = a
3  b = [7]
4  print(a)
```

[1, 2]. b is peeled off and stuck on a new list. a is untouched.

THE TIEBREAKER

Methods, index assignment and del change the list. A plain b = … only moves the name b.

a += [2] versus a = a + [2]

+=

Changes the list in place

```
1  a = [1]
2  b = a
3  a += [2]
4  print(b)
```

[1, 2]. For lists, += extends the existing list, so b sees it.

= +

Builds a new list

```
1  a = [1]
2  b = a
3  a = a + [2]
4  print(b)
```

[1]. + makes a new list, and a is moved onto it. b keeps the old one.

THE TRAP

For lists, x += y is not the same as x = x + y when another name shares the list.

Deleting through one name empties it for both

```
1  nums = [1, 2, 3]
2  vals = nums
3  del vals[1:2]
4  print(nums, len(nums))    # [1, 3] 2
5  del vals[:]
6  print(nums)               # []
```

3 del on a slice changes the list itself, so nums shrinks too.

5 del vals[:] empties the shared list. del vals would only remove the name.

A copy is shallow: inner lists are still shared

```
1  m = [[1, 2], [3]]
2  s = m[:]
3  s[0][0] = 99       # changes an inner list both share
4  s[1] = [7]         # replaces a slot in s only
5  print(m)           # [[99, 2], [3]]
6  print(s)           # [[99, 2], [7]]
```

2 The copy is a new outer list, but its elements are the same inner lists.

3 Reaching into a shared inner list changes it for both.

4 Replacing a whole slot only affects the copy.

[[0] * 3] * 3 repeats one row three times

MULTIPLYING A LIST OF LISTS

Three names for one row

```
1  grid = [[0] * 3] * 3
2  grid[0][0] = 1
3  print(grid)
```

[[1, 0, 0], [1, 0, 0], [1, 0, 0]]. Every row is the same list.

A COMPREHENSION

Three separate rows

```
1  grid = [[0] * 3 for i in range(3)]
2  grid[0][0] = 1
3  print(grid)
```

[[1, 0, 0], [0, 0, 0], [0, 0, 0]]. A new row is built each pass.

THE TRAP

* repeats references, not copies. Build rows with a comprehension or a loop.

sort() changes the list; sorted() returns a new one

LIST.SORT()

In place, returns None

```
1  a = [3, 1, 2]
2  b = a.sort()
3  print(a, b)
```

[1, 2, 3] None. a is sorted; b is None.

SORTED(LIST)

New list, original kept

```
1  a = [3, 1, 2]
2  b = sorted(a)
3  print(a, b)
```

[3, 1, 2] [1, 2, 3]. a is unchanged; b is sorted.

THE TIEBREAKER

Assigning the result of sort() is always a mistake: you get None.

min, max, sum, len, and more on sorted

```
1  nums = [4, -2, 7]
2  print(min(nums), max(nums), sum(nums), len(nums))   # -2 7 9 3
3  print(max(["apple", "Banana"]))    # apple
4  print(sorted("cab"))                # ['a', 'b', 'c']
5  print(sorted(nums, reverse=True))   # [7, 4, -2]
```

3 Strings compare by code point: lower-case a beats capital B.

4 sorted() always returns a list, even when given a string.

max([]) raises ValueError. sum([]) is 0. Sorting a mix of str and int raises TypeError.

[expression for item in sequence if condition]

```
1  squares = [x * x for x in range(4)]              # [0, 1, 4, 9]
2  threes = [x for x in range(10) if x % 3 == 0]    # [0, 3, 6, 9]
3  caps = [c.upper() for c in "abc"]                # ['A', 'B', 'C']
4  nothing = [i for i in range(-1, -2)]            # []
```

1. For each x, the expression on the left becomes one element.
2. The if filters: only items that pass it produce an element.
4. An empty range gives an empty list. range(-1, -2) counts up, so it is empty.

Every comprehension is a loop with append

THE LOOP

Four lines

```
evens = []
for n in range(6):
    if n % 2 == 0:
        evens.append(n)
```

[0, 2, 4]

THE COMPREHENSION

One line, same result

```
evens = [n for n in range(6) if n % 2 == 0]
```

[0, 2, 4]. Unlike the loop, n does not exist afterwards.

EXAM TECHNIQUE

If a comprehension confuses you, rewrite it as the loop. The result is always the same list.

A matrix: grid[row][column]

```
1  board = [[1, 2, 3],
2           [4, 5, 6]]
3  print(board[1][2])                 # 6    row 1, then column 2
4  print(len(board), len(board[0]))   # 2 3  rows, then columns
5  print([row[1] for row in board])   # [2, 5]  a whole column
6  print(board[2][0])                 # IndexError: there is no row 2
```

3 The first index picks a row, the second picks an element within it.

5 A column is not stored anywhere. Build it by taking one element from each row.

6 Two rows means valid row indexes 0 and 1.

A cube, from Python Essentials 1

```
1  rooms = [[[False for r in range(3)] for f in range(2)]
2           for b in range(2)]
3  rooms[1][0][2] = True       # building 1, floor 0, room 2
4  print(len(rooms), len(rooms[0]), len(rooms[0][0]))   # 2 2 3
5  print(rooms[1][0])          # [False, False, True]
```

1-2 Nested comprehensions: 2 buildings, each with 2 floors, each with 3 rooms.

3 Three indexes, outermost first: building, then floor, then room.

A stack is last in, first out

```
1  stack = []
2  stack.append(1)
3  stack.append(2)
4  stack.append(3)
5  print(stack.pop(), stack.pop(), stack)   # 3 2 [1]
6  queue = [1, 2, 3]
7  print(queue.pop(0), queue)               # 1 [2, 3]
```

2-5 append pushes onto the top; pop takes from the top. LIFO: last in, first out.

6-7 pop(0) takes from the front instead: first in, first out, or FIFO, like a queue.

What to carry into the exam

- b = a shares one list. [:], list() and copy() make a new one.
- Methods, index assignment, del and += change the shared list. b = [...] only moves b.
- Copies are **shallow**: inner lists are shared. [[0]*3]*3 repeats one row.
- **sort() returns None**; sorted() returns a new list.
- A comprehension is a loop with append; its variable does not survive it.

What is the output of this program?

```
1  a = [1, 2, 3]
2  b = a
3  c = a[:]
4  b[0] = 7
5  c[1] = 8
6  print(a)
```

A [7, 2, 3]

B [7, 8, 3]

C [1, 2, 3]

D [1, 8, 3]

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = [1, 2, 3]
2  b = a
3  c = a[:]
4  b[0] = 7
5  c[1] = 8
6  print(a)
```

b is the same list as a. c is a separate copy.

A `[7, 2, 3]`

b shares a's list, so the 7 shows. c is a copy, so the 8 does not.

B `[7, 8, 3]`

Treats c as another alias. [:] makes a new list.

C `[1, 2, 3]`

Treats b as a copy. Assignment never copies.

D `[1, 8, 3]`

Swaps the two: b is the alias, c is the copy.

THE SIGNAL

Label each name: alias or copy. Only changes through aliases reach the original.

What is the output of this program?

```
1  x = [1]
2  y = x
3  x += [2]
4  x = x + [3]
5  print(y, x)
```

A `[1, 2, 3] [1, 2, 3]`

B `[1] [1, 2, 3]`

C `[1, 2] [1, 2, 3]`

D `[1, 2] [1, 2]`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = [1]
2  y = x
3  x += [2]
4  x = x + [3]
5  print(y, x)
```

Line 3 extends the shared list: y sees [1, 2]. Line 4 builds a new list for x only.

A `[1, 2, 3] [1, 2, 3]`
Treats x = x + [3] as changing the shared list. It makes a new one.

B `[1] [1, 2, 3]`
Treats x += [2] as making a new list. For lists it extends in place.

C `[1, 2] [1, 2, 3]`
+= changes the shared list; = + moves x to a new list.

D `[1, 2] [1, 2]`
Ignores line 4. x itself does get the 3.

THE SIGNAL

For lists: += changes the list in place; x = x + … builds a new list and moves x.

What is the output of this program?

```
1  m = [[0, 0], [0, 0]]
2  n = m.copy()
3  n[0][0] = 5
4  n[1] = [9, 9]
5  print(m)
```

A `[[0, 0], [0, 0]]`

B `[[5, 0], [9, 9]]`

C `[[5, 0], [0, 0]]`

D `[[0, 0], [9, 9]]`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  m = [[0, 0], [0, 0]]
2  n = m.copy()
3  n[0][0] = 5
4  n[1] = [9, 9]
5  print(m)
```

copy() is shallow: m and n hold the same inner lists until one is replaced.

A `[[0, 0], [0, 0]]`
Assumes a deep copy. Inner lists are shared.

B `[[5, 0], [9, 9]]`
Treats n as an alias. Replacing n[1] only changes n.

C `[[5, 0], [0, 0]]`
Line 3 changes a shared inner list; line 4 changes n's outer list only.

D `[[0, 0], [9, 9]]`
Reverses which change reaches m.

THE SIGNAL

In a shallow copy, reaching into an inner list is shared; replacing a whole slot is not.

What is the output of this program?

```
1  nums = [3, 1, 2]
2  result = nums.sort()
3  print(result, sorted(nums, reverse=True))
```

A `[1, 2, 3] [3, 2, 1]`

B `None [3, 2, 1]`

C `None [1, 2, 3]`

D `[1, 2, 3] None`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  nums = [3, 1, 2]
2  result = nums.sort()
3  print(result, sorted(nums, reverse=True))
```

sort() sorts nums in place and returns None. sorted() returns a new list.

A `[1, 2, 3] [3, 2, 1]`
Assumes sort() returns the sorted list. It returns None.

B `None [3, 2, 1]`
sort() gives None; sorted with reverse=True gives a new descending list.

C `None [1, 2, 3]`
Ignores reverse=True.

D `[1, 2, 3] None`
Swaps which function returns None.

THE SIGNAL

sort() returns None, always. sorted() returns a list, always.

What is the output of this program?

```
1  data = [x * 2 for x in range(5) if x % 2 == 1]
2  print(data)
```

A [0, 4, 8]

B [1, 3]

C [2, 6, 10]

D [2, 6]

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  data = [x * 2 for x in range(5) if x % 2 == 1]
2  print(data)
```

range(5) is 0 to 4. The odd ones are 1 and 3. Doubled: 2 and 6.

A `[0, 4, 8]`
Keeps the even values. The test keeps the odd ones.

B `[1, 3]`
Filters correctly but forgets to double.

C `[2, 6, 10]`
Includes 5. range(5) stops at 4.

D `[2, 6]`
Filter first: 1 and 3. Then apply the expression: 2 and 6.

THE SIGNAL

Read a comprehension in order: sequence, then filter, then the expression on the left.

What is the output of this program?

```
1  grid = [[0] * 2] * 2
2  grid[1][0] = 5
3  print(grid)
```

A `[[5, 0], [5, 0]]`

B `[[0, 0], [5, 0]]`

C `[[0, 5], [0, 5]]`

D `[[0, 0], [0, 0]]`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  grid = [[0] * 2] * 2
2  grid[1][0] = 5
3  print(grid)
```

The outer * 2 repeats a reference to one inner list. Both rows are that list.

A `[[5, 0], [5, 0]]`
Both rows are the same list, so the change appears twice.

B `[[0, 0], [5, 0]]`
*Assumes two separate rows. * repeats one row.*

C `[[0, 5], [0, 5]]`
Mixes up row and column. grid[1][0] is the first element of a row.

D `[[0, 0], [0, 0]]`
The assignment does change the shared row.

THE SIGNAL

[[…]] * n makes n references to one inner list. Changing one row changes them all.

> board = [[1, 2, 3], [4, 5, 6]] has been assigned. Each expression is evaluated on its own.

Which two expressions evaluate to 6? (Choose two.)

A `board[1][2]`

B `board[2][1]`

C `board[-1][-1]`

D `max(board[0])`

◆ PAUSE AND DECIDE

Which two expressions evaluate to 6? (Choose two.)

A `board[1][2]`
Row 1 is [4, 5, 6]; element 2 is 6.

B `board[2][1]`
There is no row 2. IndexError.

C `board[-1][-1]`
Last row, last element: 6.

D `max(board[0])`
board[0] is [1, 2, 3], whose maximum is 3.

THE SIGNAL

Row first, then column. Negative indexes work at every level.

What is the output of this program?

```
1  stack = []
2  for n in [4, 5, 6]:
3      stack.append(n)
4  stack.pop()
5  stack.append(stack.pop() * 10)
6  print(stack)
```

A [4, 5, 60]

B [4, 50]

C [50, 4]

D [40, 5]

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  stack = []
2  for n in [4, 5, 6]:
3      stack.append(n)
4  stack.pop()
5  stack.append(stack.pop() * 10)
6  print(stack)
```

[4, 5, 6] -> pop removes 6 -> [4, 5] -> pop removes 5 -> append 50 -> [4, 50].

A `[4, 5, 60]`

Forgets the first pop removed the 6.

B `[4, 50]`

Two pops take 6 then 5; 5 * 10 goes back on top.

C `[50, 4]`

append adds to the end, not the front.

D `[40, 5]`

Pops from the front. pop() takes from the top: last in, first out.

THE SIGNAL

A stack only ever changes at its top: append pushes there, pop takes from there.